ValidationPanel - バリデーション結果表示パネル

1. 概要

1.1 コンポーネントの目的

ValidationPanelは、MAIML XMLのバリデーション結果を表示するモーダルダイアログです。バリデーション成功/失敗の判定、エラー・警告の詳細表示、自動補正機能を提供します。

主要機能:

- バリデーション結果サマリー表示
- エラー詳細の階層的表示
- 警告詳細の表示
- エラーチェックボックス選択による自動補正対象選択
- 自動補正実行
- 再検証機能
- 進行状態表示（検証中・補正中）

1.2 使用場面

- EditorToolbarの「バリデーション」ボタンクリック時
- 保存前の自動バリデーション実行時
- バリデーション結果を確認・補正したい時

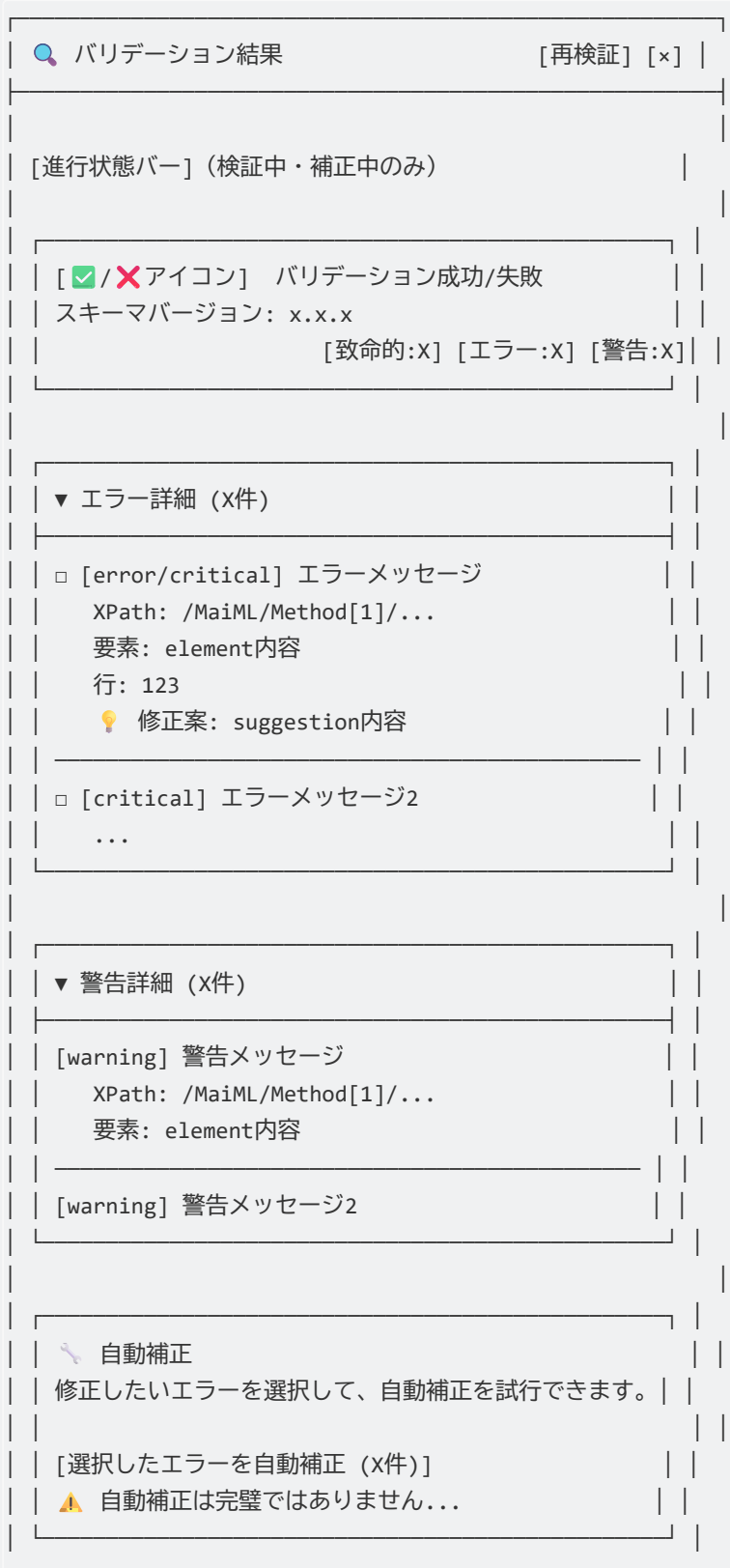
1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造



```
│                                                          │
│ [✅ MaiMLファイルは有効です]（成功時のみ）                  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 モーダルヘッダー

- **タイトル**: "🔍 バリデーション結果"
- **再検証ボタン**: ReloadOutlinedアイコン、検証中はdisabled
- **閉じるボタン**: CloseOutlinedアイコン

2.2.2 進行状態バー

- **表示条件**: isValidating または isCorrecting が true
- **要素**:
  - Progress（status="active"、showInfo=false）
  - テキスト: "🔍 検証中..." または "🔧 補正適用中..."

2.2.3 バリデーション結果サマリーカード

- **成功時**:
  - アイコン: CheckCircleOutlined（緑色、40px）
  - タイトル: "✅ バリデーション成功"
  - スキーマバージョン表示

- **失敗時**:
  - アイコン: CloseCircleOutlined（赤色、40px）
  - タイトル: "❌ バリデーション失敗"
  - スキーマバージョン表示
  - タグ表示:
    - 致命的エラー: Tag（color="error"）
    - エラー: Tag（color="error"）

    - 警告: Tag（color="warning"）

2.2.4 エラー詳細Collapse

- **ヘッダー**: "❌ エラー詳細 (X件)"
- **アイコン**: CloseCircleOutlined（赤色）
- **デフォルト状態**: 展開（activeKeys: ['errors']）
- **各エラー項目**:
    - チェックボックス（自動補正対象選択用）
    - Tag（severity: "critical" → error色、それ以外 → warning色）
    - エラーメッセージ
    - XPath（あれば）
    - 要素（あれば）
    - 行番号（あれば）
    - 💡 修正案（suggestionがあれば、青色テキスト）

2.2.5 警告詳細Collapse

- **ヘッダー**: "⚠️ 警告詳細 (X件)"
- **アイコン**: WarningOutlined（黄色）
- **各警告項目**:
    - Tag（color="warning"）
    - 警告メッセージ
    - XPath（あれば）
    - 要素（あれば）

2.2.6 自動補正カード（エラーがある場合のみ表示）

- **タイトル**: "🔧 自動補正"
- **説明**: "修正したいエラーを選択して、自動補正を試行できます。"
- **ボタン**:

- ラベル: "選択したエラーを自動補正 (X件)"
- アイコン: ToolOutlinedまたはSpin（補正中）
- disabled: selectedErrors.length === 0 || isCorrecting || isValidating
- type: primary
- block: true

- **注意テキスト**: "⚠️ 自動補正は完璧ではありません。適用後に必ず内容を確認してください。"

2.2.7 成功メッセージAlert（成功時のみ表示）

- **メッセージ**: "✅ MaiMLファイルは有効です"
- **説明**: "スキーマバリデーションに合格しました。"
- **タイプ**: success
- **showIcon**: true

---

3. 機能仕様

3.1 Props定義

```
export interface ValidationPanelProps {
  show: boolean;
  validationResult: AnalysisResult | null;
  onClose: () => void;
  onAutoCorrect?: (selectedErrors: number[]) => void;
  onRevalidate?: () => void;
  isValidating?: boolean;
  isCorrecting?: boolean;
}
```

Props説明:

Prop名	型	必須	説明
show	boolean	✅	モーダル表示状態
validationResult	AnalysisResult | null	✅	バリデーション結果

onClose	() => void	✅	モーダルを閉じるコールバック
onAutoCorrect	(selectedErrors) => void	-	自動補正実行コールバック
onRevalidate	() => void	-	再検証実行コールバック
isValidating	boolean	-	検証中フラグ（デフォルト: false）
isCorrecting	boolean	-	補正適用中フラグ（デフォルト: false）

AnalysisResult型:

```
interface AnalysisResult {
  isValid: boolean;
  version?: string;
  errors?: ErrorContent[];
  warnings?: ErrorContent[];
}

interface ErrorContent {
  severity?: 'critical' | 'error' | 'warning';
  message: string;
  path?: string;
  element?: unknown;
  line?: number;
  suggestion?: unknown;
}
```

3.2 State定義

```
const [selectedErrors, setSelectedErrors] = useState<number[]>([]);
const [activeKeys, setActiveKeys] = useState<string[]>(['errors']);
const [prevValidationResult, setPrevValidationResult] = useState<AnalysisResult | null>(null);
```

State説明:

State名	型	初期値	説明
selectedErrors	number[]	[]	自動補正対象として選択されたエラーのインデックス配列
activeKeys	string[]	['errors']	Collapseの展開状態（'errors', 'warnings'）
prevValidationResult	AnalysisResult | null	null	前回のバリデーション結果（変更検知用）

3.3 バリデーション結果変更検知

```
if (validationResult !== prevValidationResult) {
  setPrevValidationResult(validationResult);
  setSelectedErrors([]);  // 選択状態をリセット

  if (validationResult) {
    console.log('✅ [ValidationPanel V2] Validation result updated');
  }
}
```

動作:

- validationResultが変更された場合、selectedErrorsをリセット
- ESLint警告回避のため、useEffectではなく手動比較を使用

3.4 自動補正実行

3.4.1 処理フロー

```
const handleApplyCorrections = () => {
  if (selectedErrors.length === 0) {
    message.warning('補正するエラーを選択してください');
    return;
  }

  if (onAutoCorrect) {
    console.log('🔧 [ValidationPanel V2] Applying corrections for errors:', selectedErrors);
    onAutoCorrect(selectedErrors);
  }
};
```

3.4.2 UI動作

- ユーザー操作: "選択したエラーを自動補正 (X件)"ボタンをクリック
- バリデーション:

    - selectedErrors.length === 0 → message.warning表示、処理中止

- 実行: 親コンポーネント（PetriNetEditorInner）のonAutoCorrect実行
- 結果: 選択されたエラーに対して自動補正が試行される

3.5 エラーチェックボックス選択

3.5.1 処理フロー

```
const handleErrorCheckChange = (index: number, checked: boolean) => {
  if (checked) {
    setSelectedErrors(prev => [...prev, index]);
  } else {
    setSelectedErrors(prev => prev.filter(i => i !== index));
  }
};
```

3.5.2 UI動作

- ユーザー操作: エラー項目のチェックボックスをクリック
- チェックON: selectedErrorsに追加
- チェックOFF: selectedErrorsから削除
- 画面更新: ボタンラベルが"選択したエラーを自動補正 (X件)"に更新

3.6 再検証

3.6.1 処理フロー

```
{onRevalidate && (
  <Tooltip title="再検証">
    <Button
      icon={<ReloadOutlined />}
      onClick={onRevalidate}
      disabled={isValidating}
      type="text"
    />
```

```
  </Tooltip>
)}
```

3.6.2 UI動作

- ユーザー操作: ヘッダーの再検証ボタンをクリック
- 実行: 親コンポーネントのonRevalidate実行
- 状態: isValidating中はdisabled
- 結果: バリデーションが再実行され、結果が更新される

---

4. レンダリング関数

4.1 convertToString ヘルパー関数

```
const convertToString = (value: unknown): string => {
  if (value === null || value === undefined) {
    return '';
  }
  if (typeof value === 'object') {
    return JSON.stringify(value);
  }
  return String(value);
};
```

機能:

- unknown型の値を文字列に安全に変換
- element、suggestionなどの表示に使用

4.2 renderValidationSummary

機能:

- バリデーション結果サマリーカードをレンダリング

- 成功/失敗アイコン、エラー件数タグを表示

表示条件:

- validationResult === null: "バリデーション結果がありません"
- validationResult !== null: 成功/失敗サマリー

4.3 renderErrors

機能:

- エラー詳細をCollapseで表示
- 各エラー項目にチェックボックスを表示
- severity、message、path、element、line、suggestionを表示

表示条件:

- validationResult.errors.length > 0

4.4 renderWarnings

機能:

- 警告詳細をCollapseで表示
- 各警告項目をリスト表示
- path、elementを表示

表示条件:

- validationResult.warnings.length > 0

---

5. スタイル仕様

5.1 モーダル設定

```
<Modal
  width={900}
  style={{ top: 20 }}
  footer={null}
>
```

説明:

- 幅: 900px
- 上マージン: 20px
- フッター: なし（自動補正ボタンはbody内に配置）

5.2 スクロールエリア

```
<div style={{ maxHeight: 'calc(90vh - 120px)', overflow: 'auto' }}>
```

説明:

- 最大高さ: ビューポート高さの90% - 120px
- オーバーフロー: auto（スクロール表示）

5.3 エラーカードスタイル

```
style={{
  marginBottom: 16,
  backgroundColor: '#fff3e0',
  border: '1px solid #ff9800'
}}
```

5.4 警告カードスタイル

```
style={{
  marginBottom: 16
```

```
}}
```

---

6. エラーハンドリング

6.1 エラーケース

エラーケース	対処方法
補正対象エラーが未選択	message.warning表示、処理中止
validationResultがnull	"バリデーション結果がありません"表示
element/suggestionが複雑なObject	JSON.stringifyで文字列化

6.2 バリデーション

- エラー選択数が0の場合: 自動補正ボタンをdisable
- 検証中・補正中: 再検証ボタン・自動補正ボタンをdisable

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/ValidationPanel.tsx`

7.2 関連ファイル

- **型定義**: `frontend/src/types/maimlValidation.ts` (AnalysisResult, ErrorContent)

7.3 依存コンポーネント

- Ant Design:

    - Modal
    - Card
    - Typography
    - Alert
    - Tag
    - Collapse
    - Button
    - Progress
    - List
    - Checkbox
    - Space
    - Spin
    - Tooltip
    - message
- Ant Design Icons:
    - CheckCircleOutlined
    - CloseCircleOutlined
    - WarningOutlined
    - ToolOutlined
    - ReloadOutlined
    - CloseOutlined

7.4 親コンポーネント

- PetriNetEditorInner

---

8. 使用例

8.1 基本的な使用

```
<ValidationPanel
  show={showValidationPanel}
  validationResult={validationResult}
  onClose={() => setShowValidationPanel(false)}
  onAutoCorrect={handleAutoCorrect}
  onRevalidate={handleRevalidate}
  isValidating={isValidating}
  isCorrecting={isCorrecting}
/>
```

8.2 バリデーション実行フロー

```
// 1. バリデーション実行
const handleValidate = async () => {
  setIsValidating(true);
  try {
    const maimlXml = await generateMAIMLXml();
    const result = await validateMAIML(maimlXml);
    setValidationResult(result);
    setShowValidationPanel(true);
  } catch (error) {
    message.error('バリデーションに失敗しました');
  } finally {
    setIsValidating(false);
  }
};

// 2. 自動補正実行
const handleAutoCorrect = async (selectedErrors: number[]) => {
  setIsCorrecting(true);
  try {
    // 選択されたエラーに対して自動補正を実行
    await applyCorrections(selectedErrors);
    message.success('補正を適用しました');

    // 再バリデーション
    await handleValidate();
  } catch (error) {
    message.error('補正の適用に失敗しました');
  } finally {
    setIsCorrecting(false);
  }
};
```

```
// 3. 再検証
const handleRevalidate = async () => {
  await handleValidate();
};
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-01-29	Material-UI → Ant Design変換	Development Team